Author(s): dywoq <dywoq.contact@gmail.com>

Revisions:
Aug-30 2026 – Creation date

1. Overview 3

2. Goals 3

3. Versioning 4

4. Components 5

5. Specifications 6

1. Overview

Vacui is an operating system running in real mode. It respects and considers limitations of the 16-bit environment, such as limited memory and segment registers size, lack of memory protection and paging.

The initial edition is VQ/1.0_R.

It is developed in the programming languages C/Assembly.

It has a codename VQ that stands for **V**acui **Q**uarter.

2. Goals

- Establish compatibility with various programming interfaces (such as POSIX) without breaking existing VQ programs.
- Easy extensibility by putting new features into separate system modules.

3. Versioning

VQ versioning relies on editions. An edition uses the following format: “VQ/**MAJ**.**MIN_TYPE**”. Below, you can see what the keywords stand for (*X* stands for a specific roadmap, feature set, bugs etc.).

- **MAJ** → A major version number. It is increased when an edition breaks compatibility or/and adds new features.
- **MIN** → A minor version number. It is increased when an edition fixes bugs or/and finishes *X*.
- **TYPE** → An edition type. It is used to show how stable, complete and safe an edition is. An edition with the type B, A, or D is not considered stable. An edition with the type R is considered stable and published.
    - **R (Release)** → It is used when an edition fully implements *X* and bugs do not occur.
    - **B (Beta)** → It is used when an edition fully implements *X* but bugs still occur.
    - **A (Alpha)** → It is used when an edition does not implement *X* completely and bugs occur.
    - **D (Debug)** → It is used when an edition includes additional debugging features (e.g. verbose logging, registers information etc.).

4. Components

VQ has the following components: **boot loader**, **kernel** and **system modules**.

The **boot loader** is split into sector and primary stages. The sector stage initializes the boot loader environment and executes the primary stage, which executes the kernel.

The **kernel** provides common services (e.g., file, registry and memory management) to user programs. It consists of kernel subsystems. A kernel subsystem is a management unit and foundational OS component having one or several responsibilities. The kernel has the following subsystems:

- Initialization and Cleaning Routines (ICR)
- Lifetime Controller (LC)
- Memory Manager (MM)
- Registry Manager (RM)
- System Module Loader (SML)
- User Process Controller (UPC)
- Handle Manager (HM)
- Interrupt Vector Manager (IVM)
- CPU Exception Handler Installer (CEHI)
- Terminal Manager (TM)

The kernel subsystems are allowed to use others functionality. However, their dependencies must be explicitly defined in their specifications to prevent circular dependency issues.

System modules are dynamically loadable libraires. They have an entry function, which is given a specific operation type and corresponding data. System modules are loaded and executed by the SML kernel subsystem.

5. Specifications

The design workbook is split into several specifications. They give a comprehensive treatment of the VQ components. Below, you can see available specifications (all of them have a prefix “**Vacui – Design workbook:** ”):

- “Initialization and Cleaning Routines Specification”
- “Lifetime Controller Specification”
- “Memory Manager Specification”
- “Registry Manager Specification”
- “System Module Loader Specification”
- “User Process Controller Specification”
- “Handle Manager Specificaiton”
- “Interrupt Vector Manager Specification”
- “CPU Exception Handler Installer Specification”

- “Terminal Manager Specification”
- “Boot loader Specification”